esan UNIVERSITY | Ingeniería de Software

// Bootcamp AWS: de Cloud a Data Lakehouse

De los datos crudos a un pipeline automatizado

ETL, arquitectura Medallion y Spec-Driven Development

Ponente: Ricardo Fabian Uculmana Quispe

Sesión 2 · 12 nov 2026

Agenda

01 Del ejemplo al pipeline

02 Arquitectura Medallion

03 Los servicios y la arquitectura

04 Spec-Driven Development

05 Cierre

¿Qué cambia con miles de archivos?

UN ARCHIVO

La Lambda basta

Lee, transforma y escribe en segundos

→

MILES

Se queda corta

Tiempo y memoria se acaban; el el esquema cambia

→

LO QUE FALTA

Otra caja de herramientas herramientas

Paralelismo, catálogo y reintentos reintentos

Qué es un ETL

- **Extraer:** leer los datos de donde estén, tal como estén.
- **Transformar:** limpiar, tipar, deduplicar y validar.
- **Cargar:** dejarlos donde el negocio los consulta.
- Tu Lambda de la sesión 1 hacía los tres pasos, en pequeño.

ETL frente a ELT

ETL

Transformar antes de cargar

Los datos llegan ya limpios al destino. Es lo que hicimos en la sesión 1.

ELT

Cargar y transformar después

Se guarda el crudo y se transforma con la potencia del destino. Es lo que haremos hoy.

Lotes frente a eventos

POR LOTES

A una hora fija

Se procesa lo acumulado. Conviene con muchos archivos pequeños.

POR EVENTOS

Cuando llega el dato

Cada archivo dispara el proceso. Es lo que montamos montamos hoy.

El caso: pedidos de una tienda

- Eventos CSV de pedidos.
- **Duplicados:** el mismo pedido llega más de una vez.
- **Tipos inconsistentes:** importes como texto, fechas en dos formatos.
- **Campos faltantes:** pedidos sin producto o sin cliente.

02

Arquitectura Medallion

Un patrón, no un servicio

- No se contrata ni se despliega: **se decide y se respeta**.
- Organiza los datos en capas de calidad creciente.
- En AWS se implementa con S3 y con prefijos o buckets.
- Lo verás con otros nombres: raw, staging, curated.

Las tres capas

CAPA	CONTENIDO	FORMATO	EN EL CASO
Bronze	Crudo e inmutable	JSON original	**Pedidos sucios**
Silver	Limpio y tipado	Parquet particionado	**Pedidos validados**
Gold	Listo para el negocio	Parquet	**Ventas por producto**

Por qué por capas

- **Trazabilidad:** siempre puedes volver al dato original.
- **Reprocesamiento:** si la transformación falla, rehaces desde bronze.
- **Separación:** el crudo no contamina lo que consume el negocio.
- Cada capa se puede corregir sin tocar las demás.

Parquet y particiones

PARQUET

Columnar y comprimido

Lee solo las columnas que pides. Mucho más pequeño que el JSON equivalente.

PARTICIONES

Carpetas por fecha

Con dt=AAAA-MM-DD, una consulta de un día no lee el resto.

03

Los servicios y la arquitectura

Los cuatro trabajos del pipeline

CATALOGAR

Glue Catalog y Crawler

Saber qué hay y con qué esquema

→

TRANSFORMAR

Glue Jobs

Pasar de una capa a la la siguiente

→

CONSULTAR

Amazon Athena Athena

Responder preguntas preguntas con SQL

→

ORQUESTAR

EventBridge y Step Functions

Qué corre, cuándo y en en qué orden

Glue Data Catalog y Crawler

DATA CATALOG

El índice de tus datos

Guarda bases de datos y tablas: dónde están los archivos y qué columnas tienen. Los datos siguen en S3.

CRAWLER

Quien llena el índice

Recorre un prefijo de S3, deduce el esquema y las las particiones, y crea o actualiza la tabla.

AWS Glue Job

- Ejecuta **Spark** gestionado: procesamiento distribuido, sin servidores que administrar. administrar.
- Es quien hace bronze → silver: deduplica, tipa y valida.
- Escala con el volumen; arranca en minutos, no en milisegundos.
- Aquí es donde una Lambda deja de alcanzar.

Amazon Athena

- SQL directamente sobre los archivos de S3, sin cargar nada.
- Serverless: **se paga por datos escaneados**, no por tiempo encendido.
- Usa las tablas del Data Catalog para saber qué consultar.
- Con **CTAS** también transforma: así se construye la capa gold.

EventBridge y Step Functions

EVENTBRIDGE

Quien dispara

Reglas que reaccionan a eventos. Detecta el archivo nuevo en bronze y arranca el flujo.

STEP FUNCTIONS

Quien ordena

Una máquina de estados: pasos en orden, reintentos, manejo de errores y un grafo que se ve correr.

Cuándo cada transformación

OPCIÓN	CUÁNDO CONVIENE	ESCALA	EN EL BOOTCAMP
Lambda	Archivos pequeños	Limitada	**Sesión 1**
Glue Job	Volumen y escala	Alta	**Bronze → silver**
Athena CTAS	La lógica cabe en SQL	Alta	**Silver → gold**

Lo que vas a construir

Un archivo en bronze dispara el flujo; cuatro pasos después, hay una tabla de negocio.

DIAGRAMA DE ARQUITECTURA

Marco reservado · 16:9

Insertar aquí la imagen y escribir su texto alternativo

El recorrido en cuatro pasos

01 · CRAWLER

Cataloga bronze

Crea la tabla del crudo crudo

→

02 · GLUE JOB

Bronze a silver

Limpia, tipa y escribe Parquet

→

03 · CRAWLER

Cataloga silver

Registra la tabla limpia limpia

→

04 · CTAS

Silver a gold

Agrega con SQL

04

Spec-Driven Development

De improvisar a especificar

- Primero el **problema** y los requerimientos.
- Después la **arquitectura** y las decisiones.
- Al final la **implementación**, que ya está acotada.
- La spec vive en el repositorio: orienta a la persona y a la IA.

La estructura de la spec

ARCHIVO	CONTENIDO	QUÉ FIJA	SI FALTA
00–01	Contexto y requisitos	El problema	**La IA lo asume**
02	Datos y calidad	El esquema	**Tipos inventados**
03	Arquitectura	Qué se despliega	**Servicios de más**
04–05	Límites y pruebas	Qué se valida	**Nada que probar**

Por qué acelera

- Menos ambigüedad, y por tanto menos iteraciones.
- El contexto se reutiliza: Terraform, el script del job y las pruebas.
- Revisar es comparar el `plan` contra la spec, no leerlo a ciegas.
- Un cambio se hace editando la spec, no reescribiendo.

Lo que no resuelve

- La IA inventa propiedades y APIs que no existen.
- Una spec ambigua produce resultados ambiguos.
- No elimina la revisión técnica: solo la enfoca.
- Por eso se valida con documentación oficial y con `terraform plan`.

Lo que te llevas de la teoría

01 Un ETL que crece se organiza por capas: bronze, silver y gold.

02 Seis servicios, cuatro trabajos: catalogar, transformar, consultar y orquestar.

03 La especificación es la fuente de verdad; el código es su consecuencia.

Referencias

Amazon Web Services. (s. f.-a). *AWS Certified Data Engineer – Associate (DEA-C01): Exam guide*. https://docs.aws.amazon.com/aws-certification/latest/data-engineer-associate-01/data-engineer-associate-01.html

Amazon Web Services. (s. f.-b). *AWS Glue crawlers*. https://docs.aws.amazon.com/glue/latest/dg/add-crawler.html

Amazon Web Services. (s. f.-c). *Creating a table from query results (CTAS)*. https://docs.aws.amazon.com/athena/latest/ug/ctas.html

Amazon Web Services. (s. f.-d). *Manage AWS Glue jobs with Step Functions*. https://docs.aws.amazon.com/step-functions/latest/dg/connect-functions/latest/dg/connect-glue.html

Amazon Web Services. (s. f.-e). *Using EventBridge with Amazon S3 Event Notifications*. https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventBridge.html

Amazon Web Services. (s. f.-f). *What is the AWS Glue Data Catalog?* https://docs.aws.amazon.com/glue/latest/dg/catalog-and-crawler.html crawler.html

esan UNIVERSITY

Ahora lo construimos

¿Dudas antes de empezar?

A

Anexos

Material de respaldo para preguntas

El mapa de DEA-C01

DOMINIO	PESO	SESIÓN 1	HOY
Ingesta y transformación	34 %	S3 y Lambda	**Glue Job y CTAS**
Gestión de almacenes	26 %	Formatos en S3	**Catálogo y Medallion**
Operaciones y soporte	22 %	CloudWatch Logs	**Step Functions**
Seguridad y gobierno	18 %	IAM y mínimo privilegio	**Roles de servicio**

¿Un bucket o varios?

UN BUCKET POR CAPA

Lo del laboratorio

Más claro de explicar y evita que la salida vuelva a disparar la entrada.

UN BUCKET CON PREFIJOS

Lo habitual en producción

Menos recursos que gestionar; hay que filtrar los los eventos con cuidado.

Lo que queda fuera

- **Streaming:** Kinesis y datos en tiempo real.
- **DynamoDB:** fuera del alcance de este bootcamp.
- **Orquestación programada:** Airflow o MWAA, y los lotes por horario.
- **Calidad de datos:** Glue Data Quality y los contratos de datos.

Glosario rápido

- **CTAS:** CREATE TABLE AS SELECT; SQL que escribe datos nuevos.
- **Partición:** carpeta que acota lo que la consulta lee.
- **State machine:** la definición de pasos de Step Functions.
- **Data scanned:** lo que Athena lee y cobra en cada consulta.